**UNIVERSIDAD TECNOLÓGICA DE PANAMÁ**
**FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES**
**DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS**
**LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN**

# Dimensionamiento y despliegue de Envoy + Cassandra 5 en un host de 6 GB de RAM y 3 vCPU con Docker

**Asignatura:**
Sistemas Operativos

**Asignación:**
Proyecto #1: Cálculo y despliegue de recursos de memoria y procesos para servicios en producción

**Caso de estudio:**
CS06 · Envoy + Cassandra 5 (plataforma de telemetría IoT)

**Profesor:**
Prof. Isacio Tamayo

**Grupo de trabajo:**
Grupo 2

**Integrantes:**
James Tack
Carlos Robles
Heidy Urriola

**Grupo:**
1IL143

**Fecha de entrega:**
15 de octubre de 2026

2do Semestre 2026

## De qué se trata el proyecto

El proyecto pide dimensionar, calcular y desplegar un entorno de producción pequeño con 6 GB de RAM y 3 vCPU, y comprobar en vivo que los números de la tabla coinciden con lo que el sistema operativo realmente aplica. A nuestro grupo le correspondió el caso de estudio 6, una plataforma de telemetría IoT que recibe 50 000 escrituras por segundo de 10 000 dispositivos concurrentes (200 ms de sesión media, relación lectura:escritura de 1:1). El servidor de entrada es Envoy y la base de datos es Cassandra 5.

Elegimos la vía de contenedores Docker, así que el overhead del host es del 25 % de la RAM (1.5 GB) y quedan 4.5 GB para los servicios. Todo corrió en la máquina virtual del curso (vm-sistemas-operativos, Ubuntu 18.04 con Docker 24.0.2 y Compose 2.18.1), que ampliamos de 2 CPU y 2 GB a 3 CPU y 6 GB para que reproduzca el escenario del enunciado.

El documento sigue el orden de los entregables, que son la tabla de recursos, la justificación técnica y la demostración en vivo con su prueba de estrés. Cada comando que aparece en las capturas se ejecutó dentro de la VM por SSH, y los archivos de configuración están en el anexo B.

## Datos del caso y vía elegida

**Tabla 1. Parámetros de partida**

| Dato | Valor | Fuente |
|---|---|---|
| Hardware base | 6 GB de RAM y 3 vCPU | Enunciado |
| Vía de despliegue | Contenedores Docker (overhead del 25.0 %) | Elección del grupo |
| Tráfico | 50 000 escrituras/s, 10 000 dispositivos, sesión de 200 ms | Guía, CS06 |
| Relación lectura:escritura | 1:1 (cargado de escrituras) | Guía, CS06 |
| mem_limit de Envoy | 0.60 GB con 0.75 vCPU | Guía, CS06 |
| Cassandra | max_conexiones = 150; costo_por_conexion = 3 MB; margen = 275 MB | Guía, CS06 |
| Memoria web nativa (referencia) | 0.25 GB | Guía, CS06 |

## Entregable 1. Tabla de recursos

### Procedimiento de cálculo

Primero se resta el overhead de los 6 GB y después se reparte lo que queda, tal como sugiere la pista del enunciado. Con Docker el sistema operativo y el daemon consumen el 25 %, es decir 6 × 0.25 = 1.50 GB, de modo que la RAM libre para servicios es 6 − 1.50 = 4.50 GB. Envoy recibe los 0.60 GB que fija la guía y Cassandra se queda con el resto, es decir 4.50 − 0.60 = 3.90 GB. De esos 3.90 GB hay que apartar lo que cuestan las conexiones (150 × 3 MB = 450 MB, o 0.44 GB) y el margen de seguridad (275 MB, o 0.27 GB). Lo que sobra es la memoria efectiva que Cassandra puede dedicar a heap, memtables y cachés, que resulta de 3.90 − 0.44 − 0.27 = 3.19 GB. Para convertir de MB a GB se usó 1 GB = 1024 MB.

En CPU, Envoy recibe 0.75 vCPU y Cassandra el resto, 3 − 0.75 = 2.25 vCPU. Con una cuota inferior a una CPU completa, Envoy necesita un único worker (⌈0.75⌉ = 1, opción --concurrency 1). Por la ley de Little, 50 000 escrituras/s × 0.2 s dan 10 000 conexiones simultáneas, que es justo el número de dispositivos del enunciado. Con un solo worker, esas 10 000 conexiones caen en un único hilo de eventos. Hacia Cassandra, Envoy agrupa todo en un máximo de 150 conexiones, igual que el límite del motor de datos.

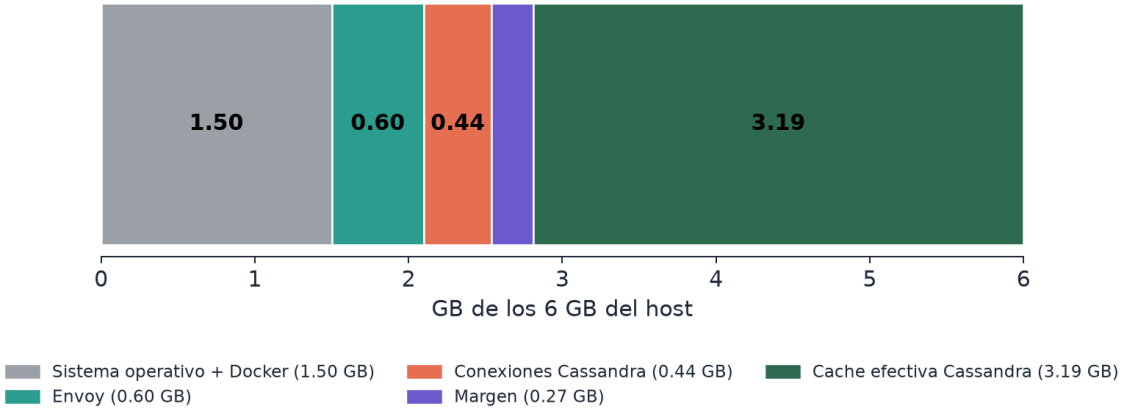



*Figura 1. Reparto de los 6 GB del host en la vía Docker.*

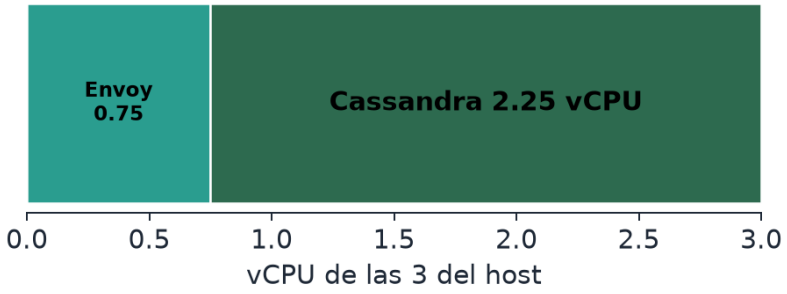



*Figura 2. Reparto de las 3 vCPU entre Envoy y Cassandra.*

## Tabla de recursos de la vía elegida

**Tabla 2. Tabla de recursos, caso CS06 (6 GB, 3 vCPU)**

| Parámetro / recurso | Docker (vía elegida) | Nativo (referencia) |
|---|---|---|
| Arquitectura elegida para el demo | [X] Docker | [ ] Nativo |
| Overhead SO / Docker daemon (GB) | 1.50 | 0.81 |
| RAM libre para servicios (GB) | 4.50 | 5.19 |
| mem_limit o RAM asignada a Envoy (GB) | 0.60 (614 MiB) | 0.25 |
| mem_limit o RAM asignada a Cassandra (GB) | 3.90 (3 993 MiB) | 4.94 |
| vCPU asignadas a Envoy | 0.75 | sin cuota (3 compartidas) |
| vCPU asignadas a Cassandra | 2.25 | sin cuota (3 compartidas) |
| Workers / hilos del servidor web | 1 worker (--concurrency 1) | 1 |
| Conexiones sostenidas por worker | 10 000 hacia los clientes; 150 hacia | igual |

| | Cassandra | |
|---|---|---|
| Conexiones de Cassandra (150 × 3 MB) | 0.44 GB | 0.44 GB |
| Margen (275 MB) | 0.27 GB | 0.27 GB |
| Caché efectiva de Cassandra (GB) | 3.19 | 4.23 |

La columna nativa sirve solo de comparación. Con instalación nativa el overhead es 13.5 % (0.81 GB), la RAM libre sube a 5.19 GB y, si Envoy usa 0.25 GB, Cassandra obtiene 4.94 GB y una caché efectiva de 4.23 GB. La diferencia de 1.04 GB frente a Docker es lo que cuesta el aislamiento.

El heap de la JVM se fijó en 2 GiB (-Xms = -Xmx). Dentro de la caché efectiva de 3.19 GB quedan además 768 MiB de memtables fuera del heap y 384 MiB de caché de archivos, para un total de 3.13 GiB comprometidos y unos 60 MiB de holgura. Esa distribución es una decisión nuestra; el enunciado solo exige la cifra total.

## Despliegue paso a paso

### Paso 1. Preparar la máquina virtual

Antes de arrancar se cambió la VM de 2 CPU y 2048 MB a 3 CPU y 6144 MB con VBoxManage modifyvm. Dentro de la VM, nproc devuelve 3 y free -m muestra 5 960 MB, que son los 6 GB menos lo que VirtualBox reserva para el firmware virtual. Docker y Compose ya venían del aprovisionamiento con Vagrant del laboratorio 1.

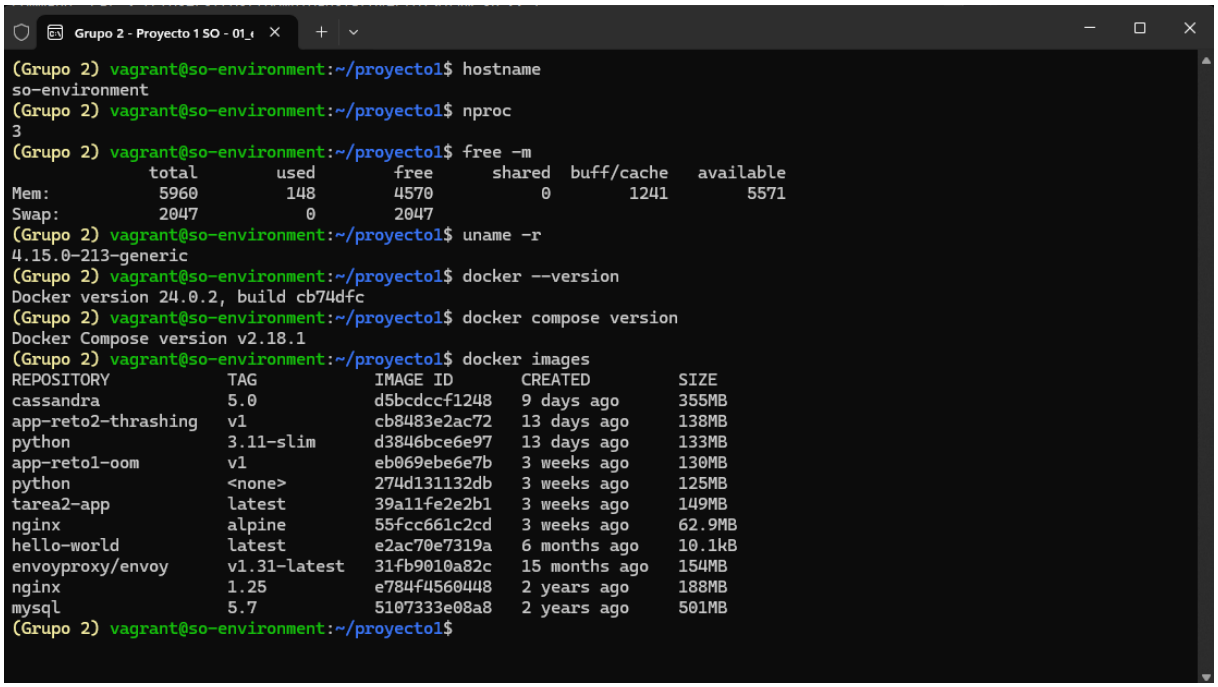



*Figura 3. La VM con 3 CPU, 5 960 MB de RAM, Docker 24.0.2 y Compose 2.18.1.*

## Paso 2. Archivos del proyecto

El despliegue lo componen cuatro archivos: docker-compose.yml con los límites, un Dockerfile que parte de cassandra:5.0 y ajusta cassandra.yaml, envoy.yaml con el listener TCP del puerto 9042 y el límite de 150 conexiones, y los scripts de monitoreo y estrés. Los mem_limit se expresaron en MiB (614 y 3 993) porque Docker no admite fracciones de GB.

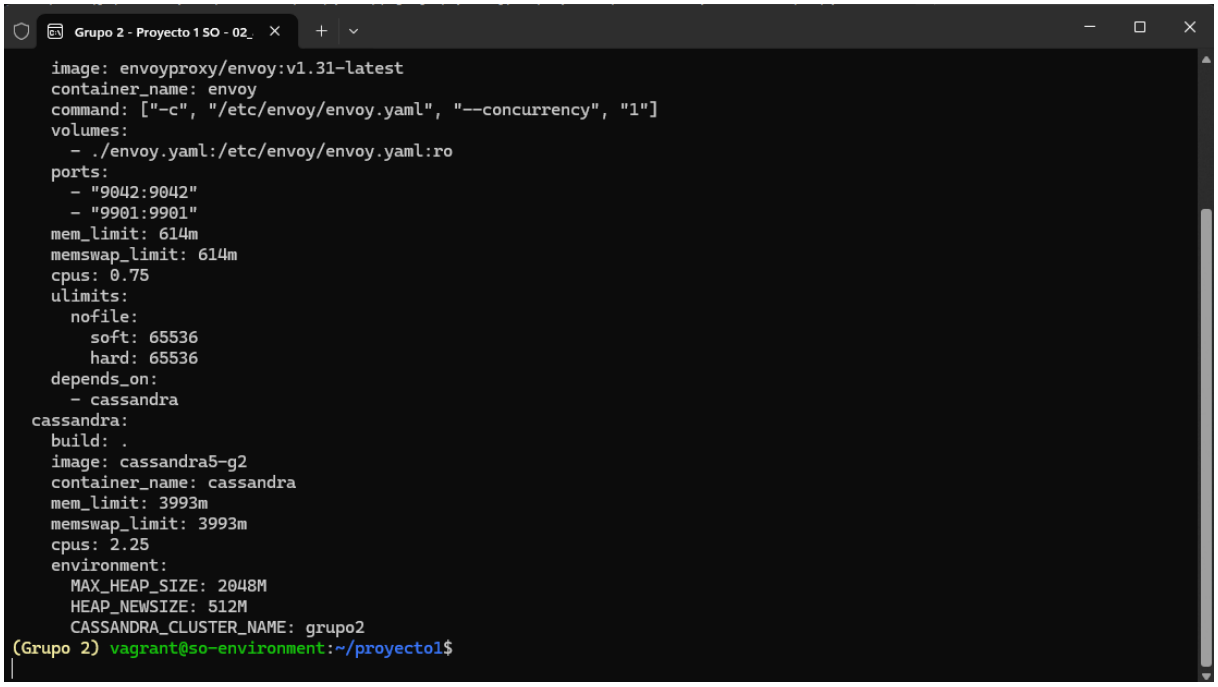



*Figura 4. Archivos del proyecto y docker-compose.yml con mem_limit, memswap_limit y cpus.*

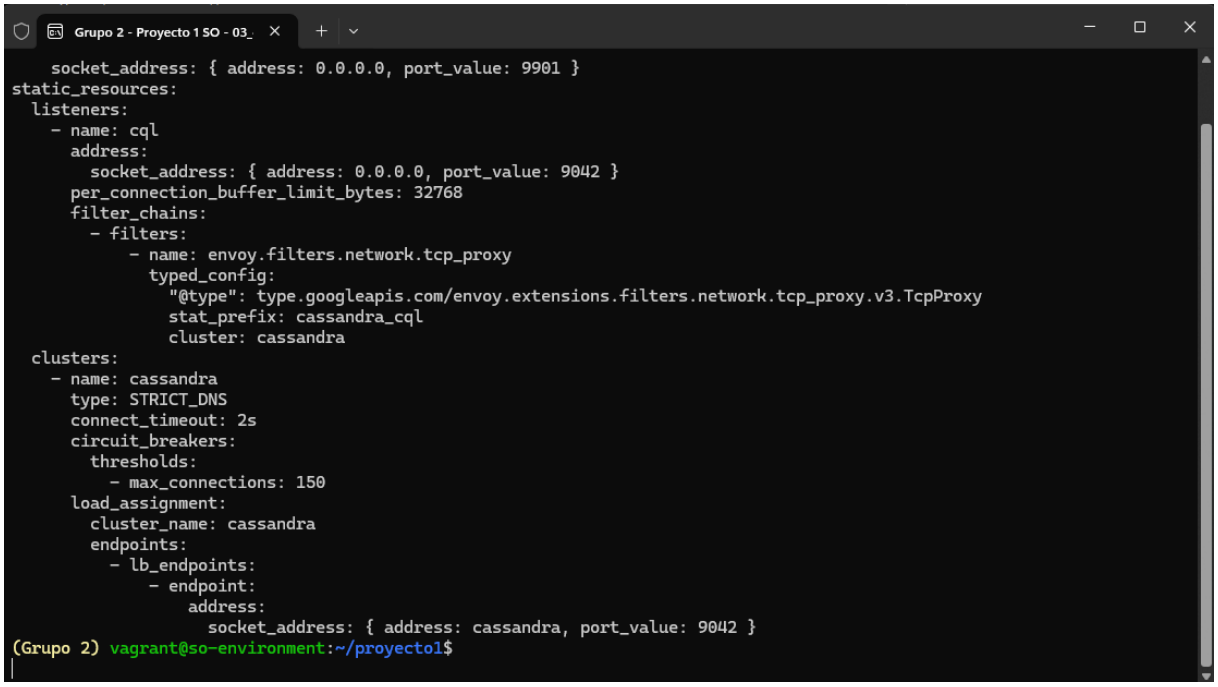



*Figura 5. envoy.yaml: listener 9042, cluster cassandra y circuit breaker de 150 conexiones.*

En el compose, memswap_limit vale lo mismo que mem_limit. Con eso el contenedor no puede usar swap, de modo que al agotar su memoria no se degrada con intercambio sino que dispara el OOM Killer de su cgroup, que es el comportamiento que se quiere observar y controlar.

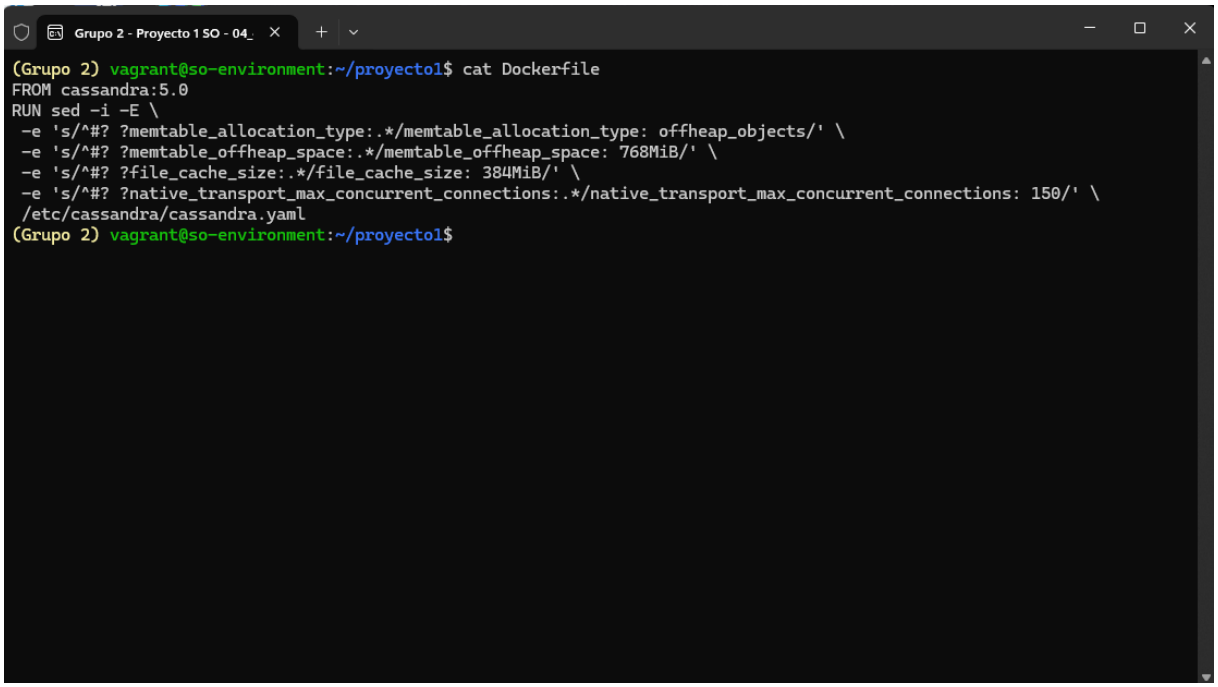



*Figura 6. Dockerfile: sed sobre cassandra.yaml para offheap_objects, 768 MiB, 384 MiB y 150 conexiones.*

## Paso 3. Construir y levantar

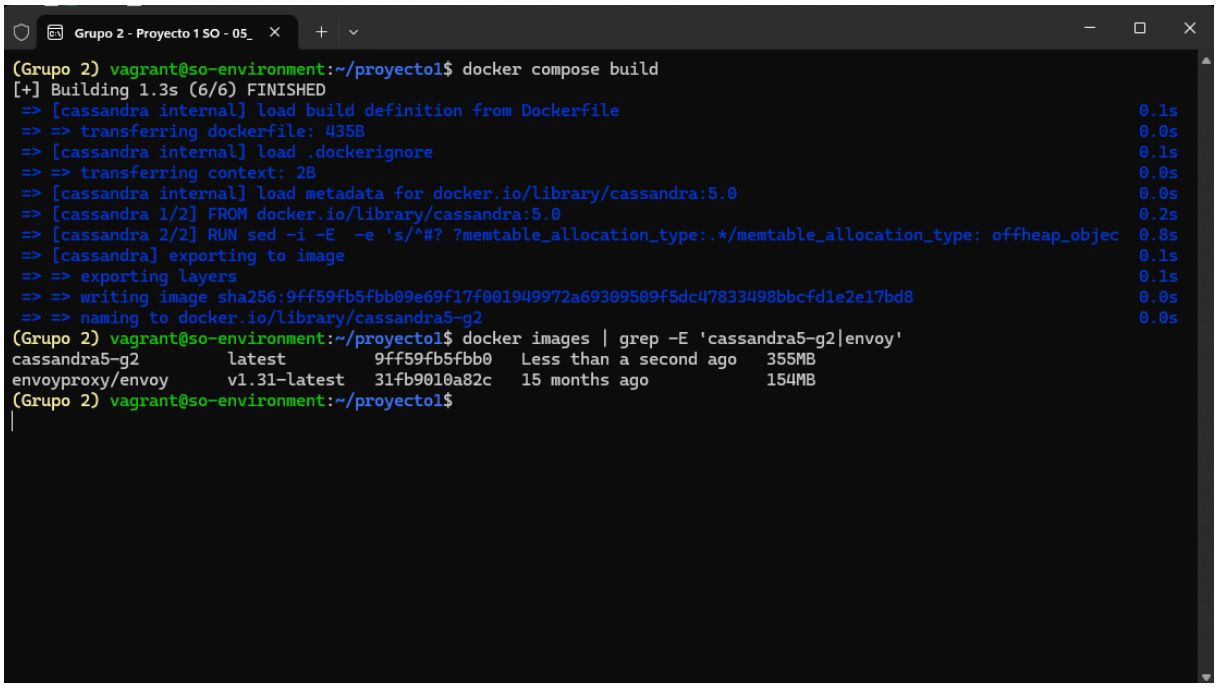



*Figura 7. docker compose build: la imagen cassandra5-g2 queda de 355 MB.*

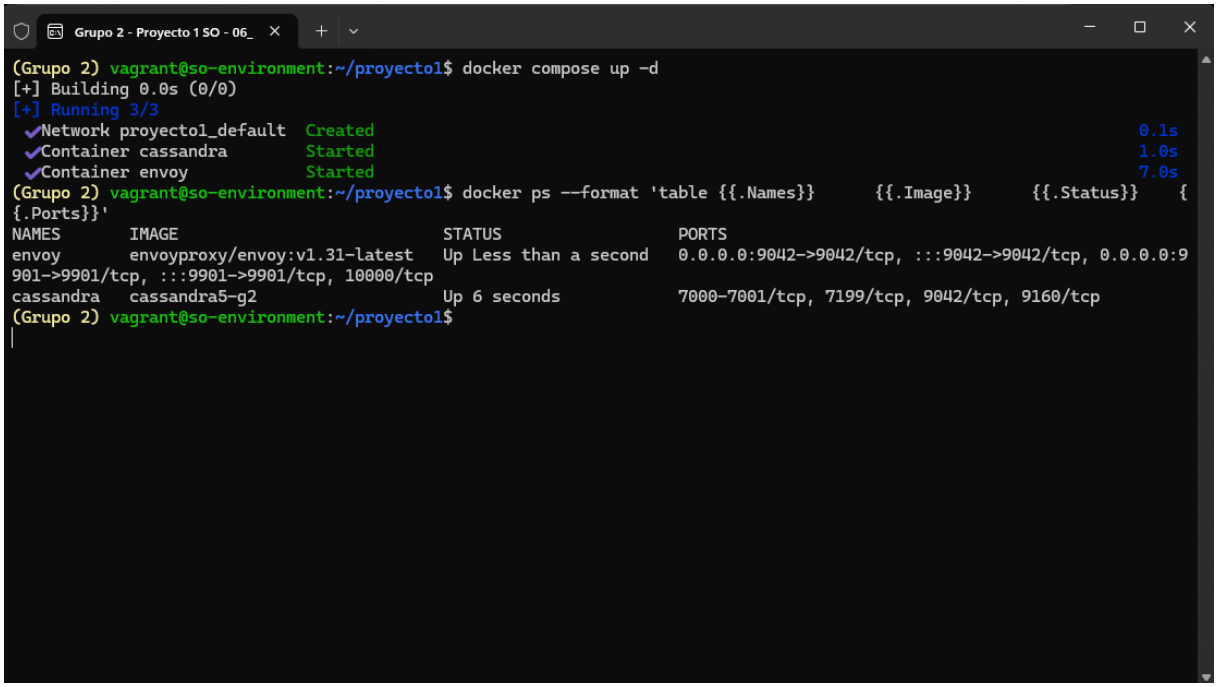



*Figura 8. docker compose up -d: contenedores envoy y cassandra en ejecución.*

Cassandra tardó unos minutos en llegar al estado UN (Up/Normal), algo normal en una JVM con 2 GiB de heap sobre una VM sin aceleración por hardware. Además, para comprobar que Envoy realmente enruta, se ejecutó cqlsh desde el contenedor de Cassandra apuntando al nombre envoy:9042, de modo que la consulta pasa por el proxy.

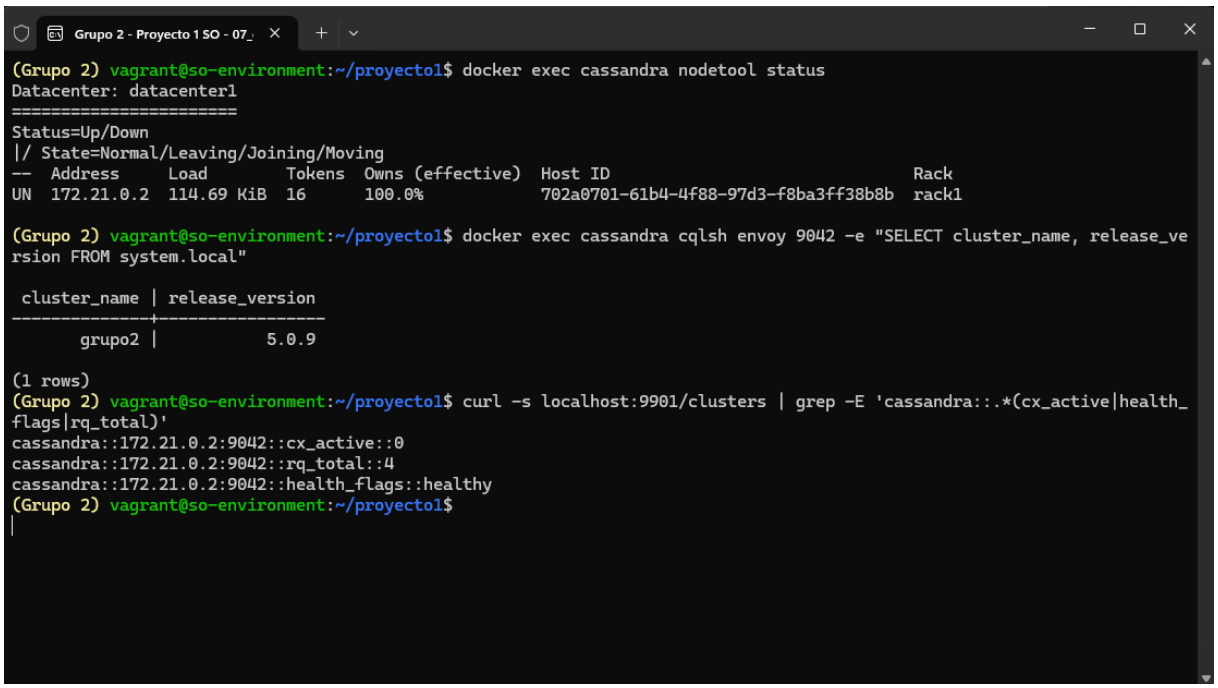



*Figura 9. nodetool status en UN y consulta CQL a través de Envoy (cluster grupo2, versión 5.0.9).*

## Entregable 3. Demostración en vivo

### Los límites aplicados coinciden con la tabla

Se comparó cada fila de la tabla con lo que informa el sistema. En docker inspect, Memory de Envoy es 643 825 664 bytes (614 MiB) y NanoCpus 750 000 000 (0.75 vCPU). Para Cassandra son 4 186 963 968 bytes (3 993 MiB) y 2 249 999 872 NanoCpus (2.25 vCPU). El cgroup v2 del host confirma los mismos valores: memory.max, memory.swap.max en 0 y cpu.max con cuotas de 75 000 y 224 999 microsegundos por periodo de 100 000.

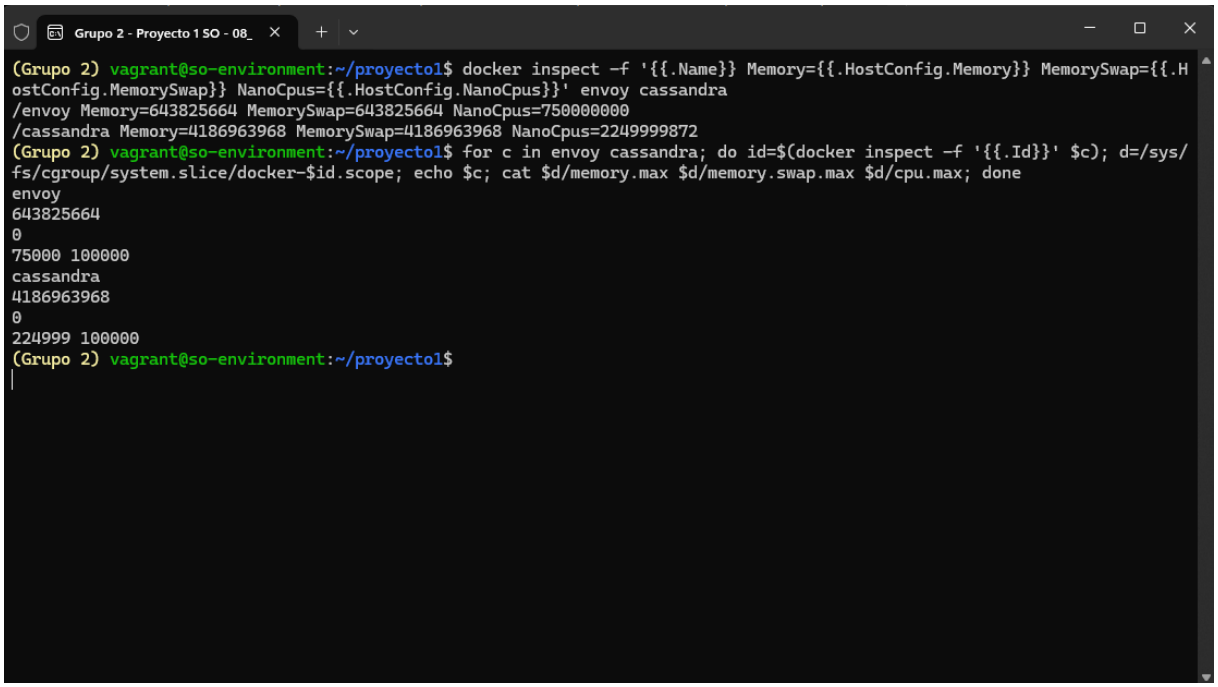



*Figura 10. docker inspect y archivos del cgroup v2 (memory.max, memory.swap.max y cpu.max).*

La configuración interna también coincide. cassandra.yaml tiene memtable_offheap_space en 768 MiB, file_cache_size en 384 MiB y native_transport_max_concurrent_connections en 150. La línea de comandos de la JVM lleva -Xms2048M y -Xmx2048M, y el servidor de administración de Envoy informa concurrency igual a 1.

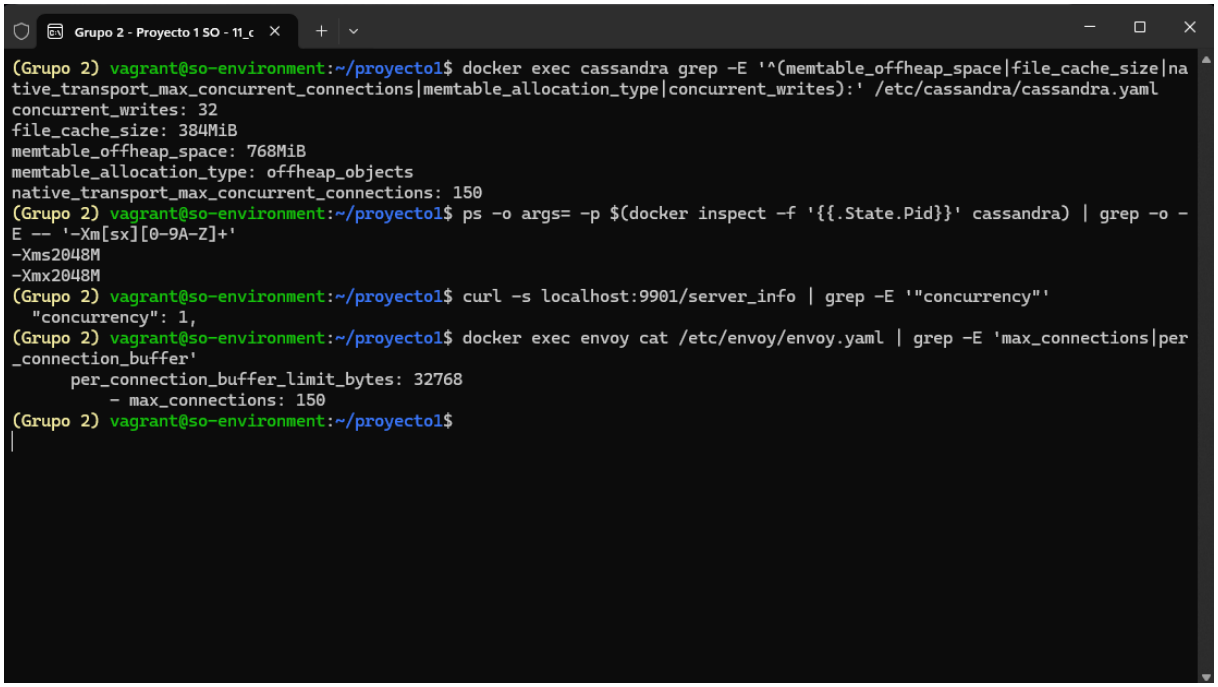



*Figura 11. Configuración real de Cassandra y Envoy leída desde dentro de los contenedores.*

## Estado en reposo

Con los servicios inactivos, docker stats marca 2.275 GiB de 3.899 GiB para Cassandra (58 %) y 12.3 MiB de 614 MiB para Envoy (2 %). free -m reporta 2 510 MB usados y 3 207 MB disponibles, y el swap del host sigue en cero.

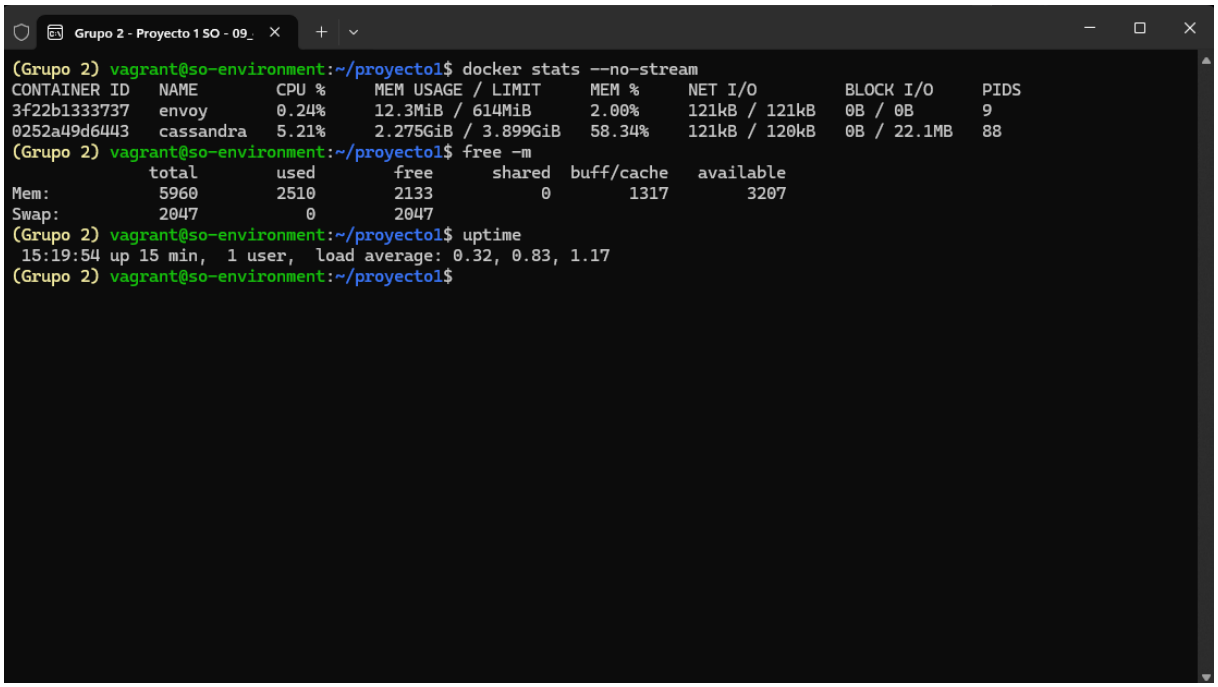



*Figura 12. docker stats y free -m con la infraestructura en reposo.*

ps aux muestra dos procesos principales, la JVM de Cassandra (PID 7055, 2.4 GB de RSS, 76 hilos) y un único proceso envoy lanzado con --concurrency 1. Envoy tiene 9 hilos de sistema, de los cuales uno es el worker de datos y el resto atiende administración, DNS y señales.

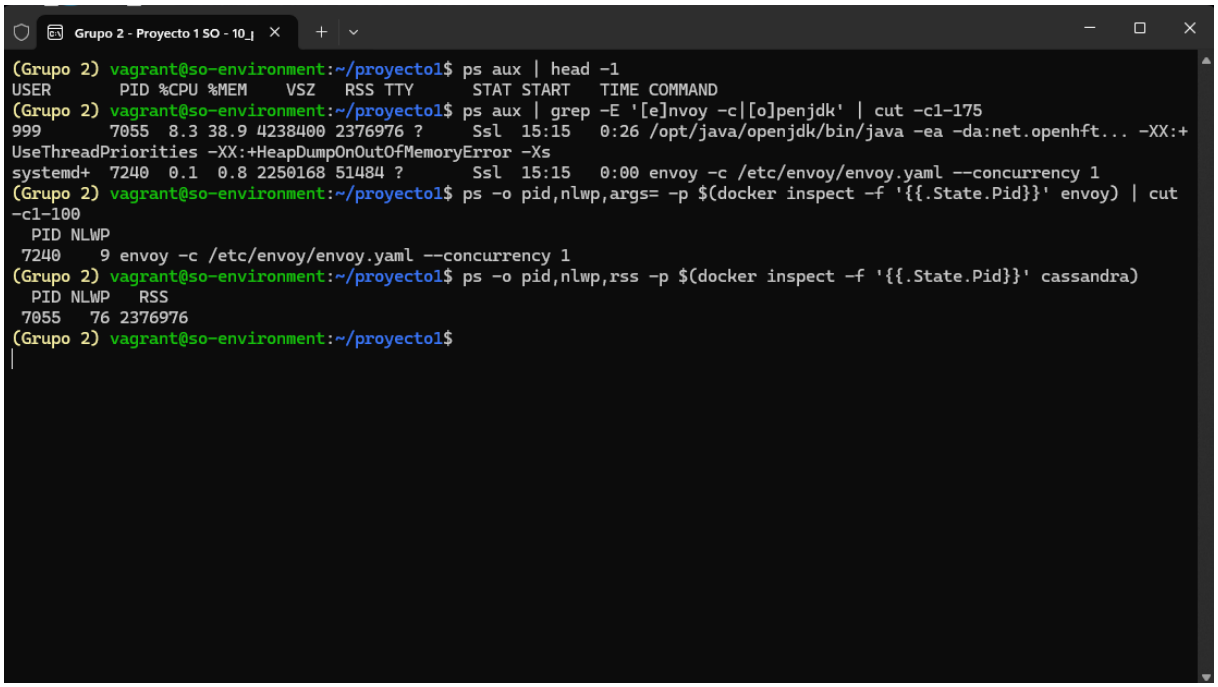



*Figura 13. Procesos de Envoy y Cassandra con su número de hilos (NLWP).*

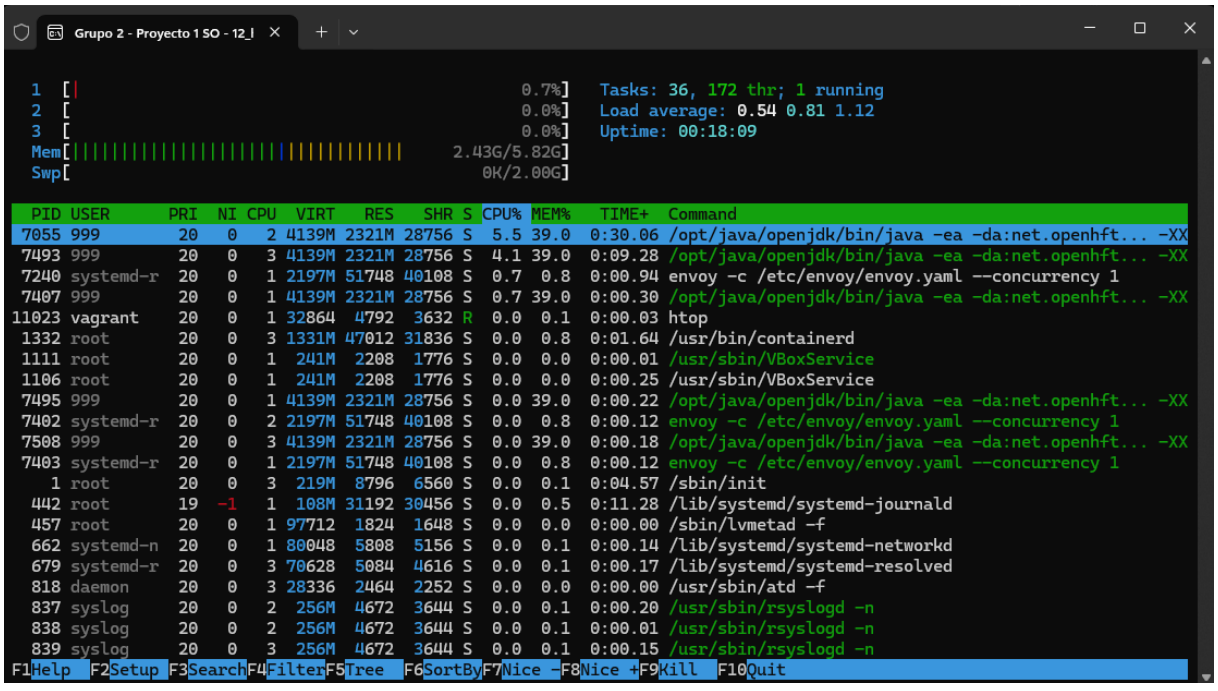



*Figura 14. htop en reposo: 3 CPU casi libres y 2.43 GB de 5.82 GB en uso.*

## Prueba de estrés controlada

### Diseño

La carga la generó cassandra-stress en un contenedor aparte (400 MB de memoria y 0.5 CPU) escribiendo a través de Envoy. Se subió la concurrencia por etapas de 40 s con 8, 32, 64 y 128 hilos, y durante cada etapa un script (monitor.sh) muestreó cada 4 s la memoria y CPU de ambos contenedores, las conexiones activas de Envoy hacia Cassandra, la memoria libre del host y el swap. Después de cada etapa se revisaba si algún contenedor había sido muerto por el OOM Killer y la prueba se detenía si Cassandra superaba el 95 % de su límite. Esa condición nunca se cumplió.

### Resultados

**Tabla 3. Resultados por etapa (cassandra-stress, 40 s por etapa)**

| Hilos | Op/s | Latencia media (ms) | p99 (ms) | Errores | Cassandra RAM (GiB) | Cassandra CPU máx. (%) | RAM libre mín. (MiB) | Swap | OOM |
|---|---|---|---|---|---|---|---|---|---|
| 8 | 1 757 | 4.5 | 54.6 | 0 | 2.48 | 101 | 1 573 | 0 | no |
| 32 | 3 862 | 8.2 | 66.1 | 0 | 2.49 | 93 | 1 519 | 0 | no |
| 64 | 1 579 | 39.7 | 396.4 | 0 | 2.55 | 113 | 1 373 | 0 | no |
| 128 | 6 985 | 17.8 | 81.6 | 0 | 2.51 | 71 | 1 510 | 0 | no |

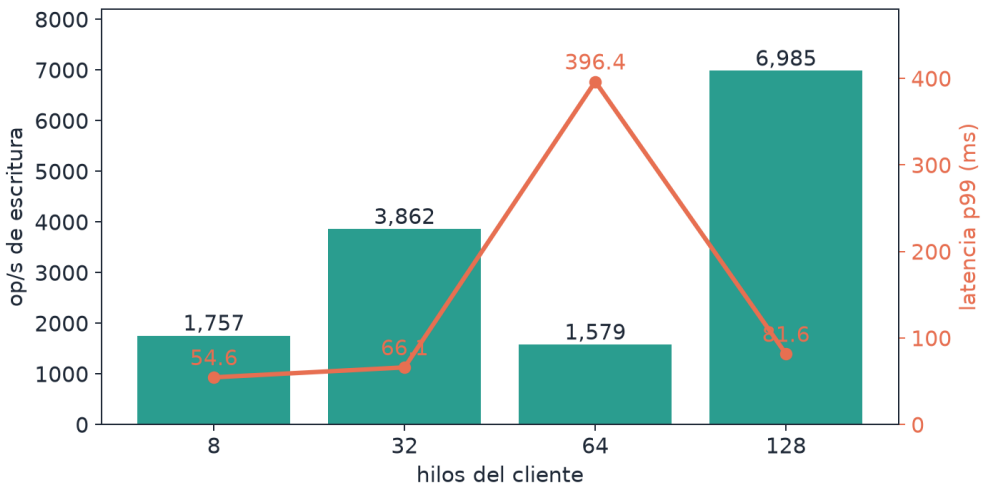



*Figura 15. Escrituras por segundo y latencia p99 según los hilos del cliente.*

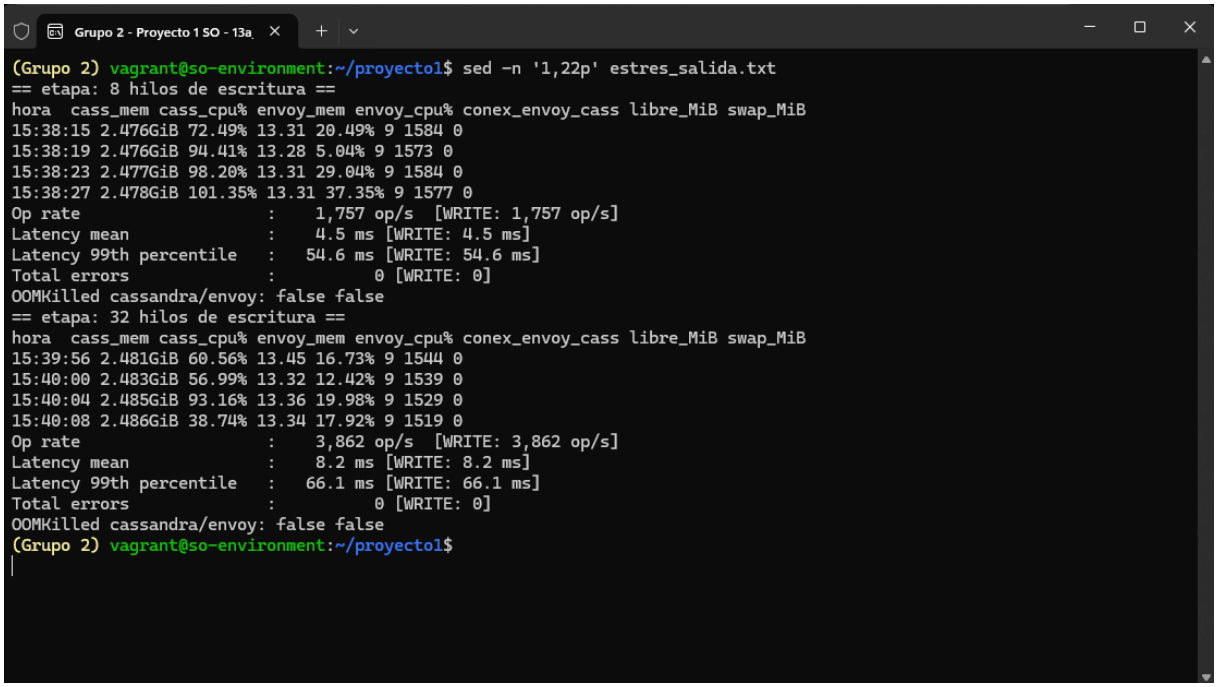



*Figura 16. Monitoreo de las etapas de 8 y 32 hilos.*

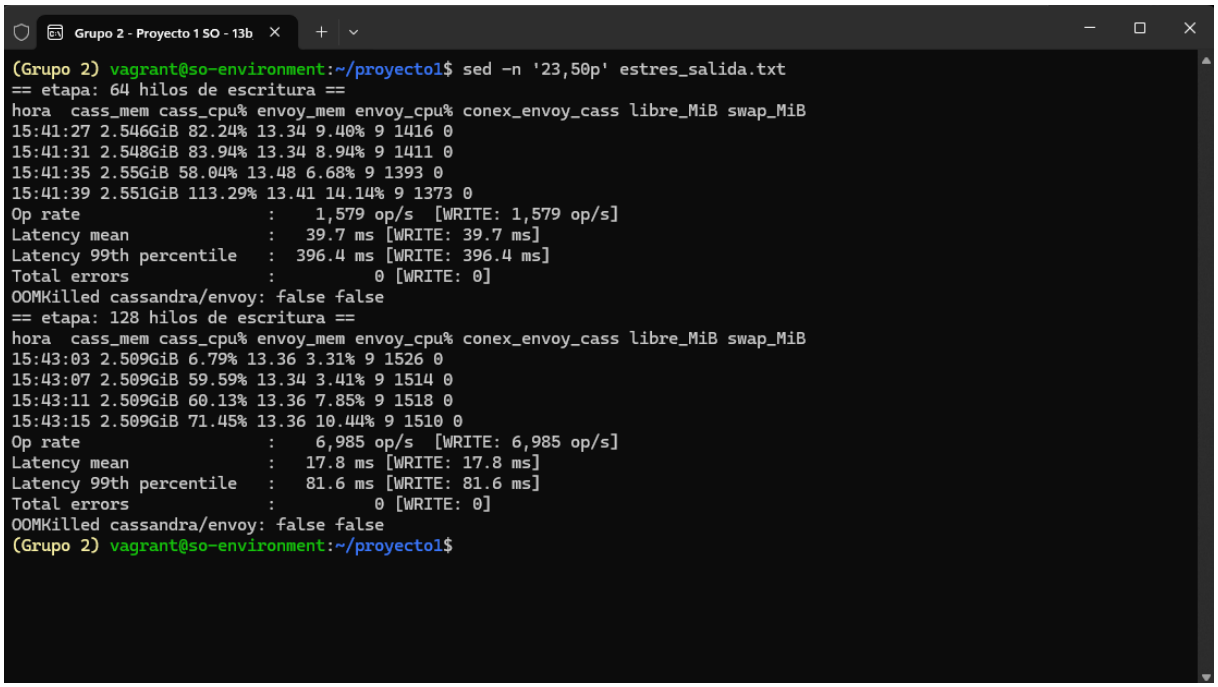



*Figura 17. Monitoreo de las etapas de 64 y 128 hilos.*

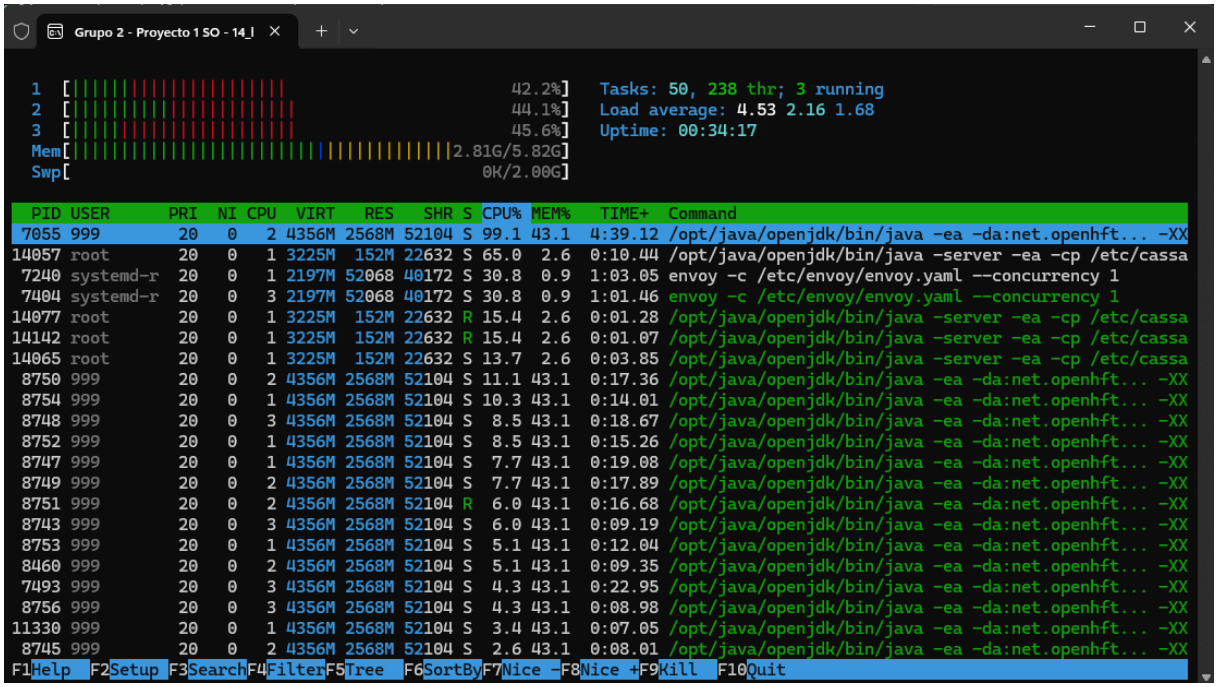



*Figura 18. htop durante la carga: las tres CPU al 42-46 %, Cassandra en 99 % y Envoy en 31 %.*

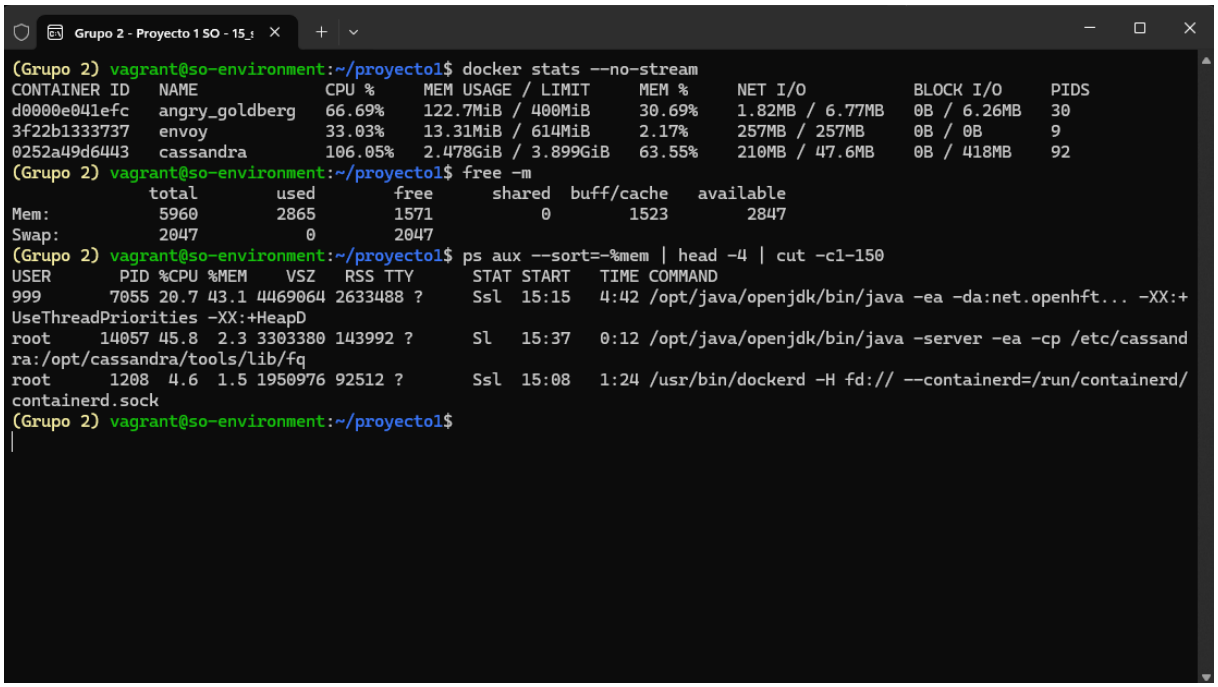



*Figura 19. docker stats bajo carga: Cassandra 106 % de CPU y 63.5 % de su memoria, Envoy 33 %.*

## Lectura de los resultados

Ninguno de los límites duros llegó a saturarse. Cassandra usó como máximo 2.55 GiB de sus 3.90 GiB (65 %), Envoy no pasó de 13.5 MiB de sus 614 MiB, el host conservó al menos 1 373 MiB libres y el swap quedó en cero en todas las etapas. En CPU, el pico de Cassandra fue 113 % sobre una cuota de 225 % y el de Envoy 37 % sobre una cuota de 75 %. Por eso no hubo ningún OOMKilled ni errores en las 4 etapas.

Sin embargo, la etapa de 64 hilos es una excepción, donde el rendimiento cayó a 1 579 op/s y el p99 subió a 396 ms. No tenemos una medición que aísle la causa. La etapa siguiente, con el doble de hilos, rindió 6 985 op/s, lo que indica que el techo no es del servicio sino variable del entorno: la VM corre sobre Hyper-V con emulación (NEM), el cliente de carga comparte las tres vCPU con ambos contenedores, y el anfitrión Windows también atiende otras tareas. Lo dejamos reportado así, sin atribuirlo a Envoy ni a Cassandra.

Por otra parte, los 50 000 escrituras/s del escenario no se alcanzaron, ya que el máximo medido fue 6 985 op/s, un 14 % de la meta, con un solo cliente de 0.5 CPU. El dimensionamiento de memoria y CPU es consistente, pero la capacidad de 50 000 op/s no se verificó en esta VM.

## Prueba del límite de conexiones

El segundo límite que sí se alcanzó fue el de conexiones. Se abrieron 50, 100, 150 y 200 conexiones TCP al puerto 9042 de Envoy. El contador upstream_cx_active subió hasta 150 y se quedó ahí; las 50 conexiones restantes sumaron en upstream_cx_overflow. Es decir, el circuit breaker del cluster hizo cumplir el número que calculamos en la tabla y Cassandra no recibió más de 150 clientes.

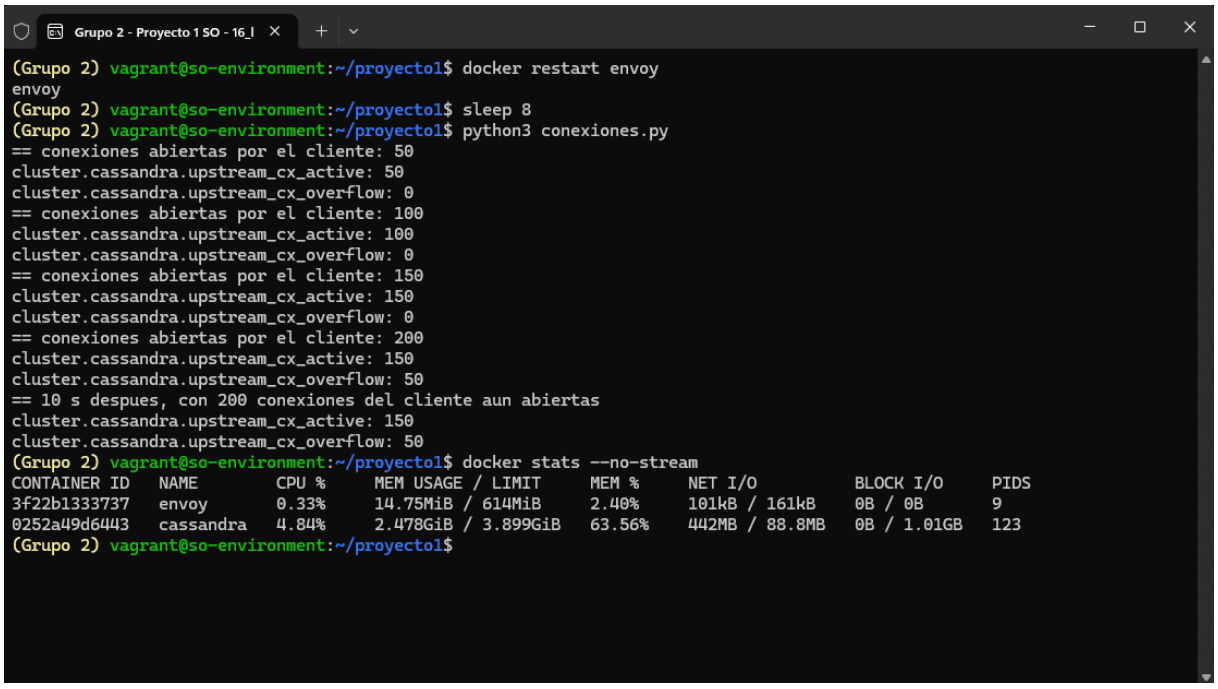



*Figura 20. Conexiones activas y desbordadas en Envoy al subir de 50 a 200 clientes.*

## Comprobación del OOM Killer

Para ver el OOM Killer sin poner en riesgo los servicios, se lanzó un contenedor desechable con 64 MB que intenta reservar 300 MB. El kernel lo mató dentro de su cgroup (código de salida 137, OOMKilled=true) y dmesg registra «Memory cgroup out of memory: Kill process 17098 (python)». Envoy y Cassandra siguieron arriba durante toda la prueba.

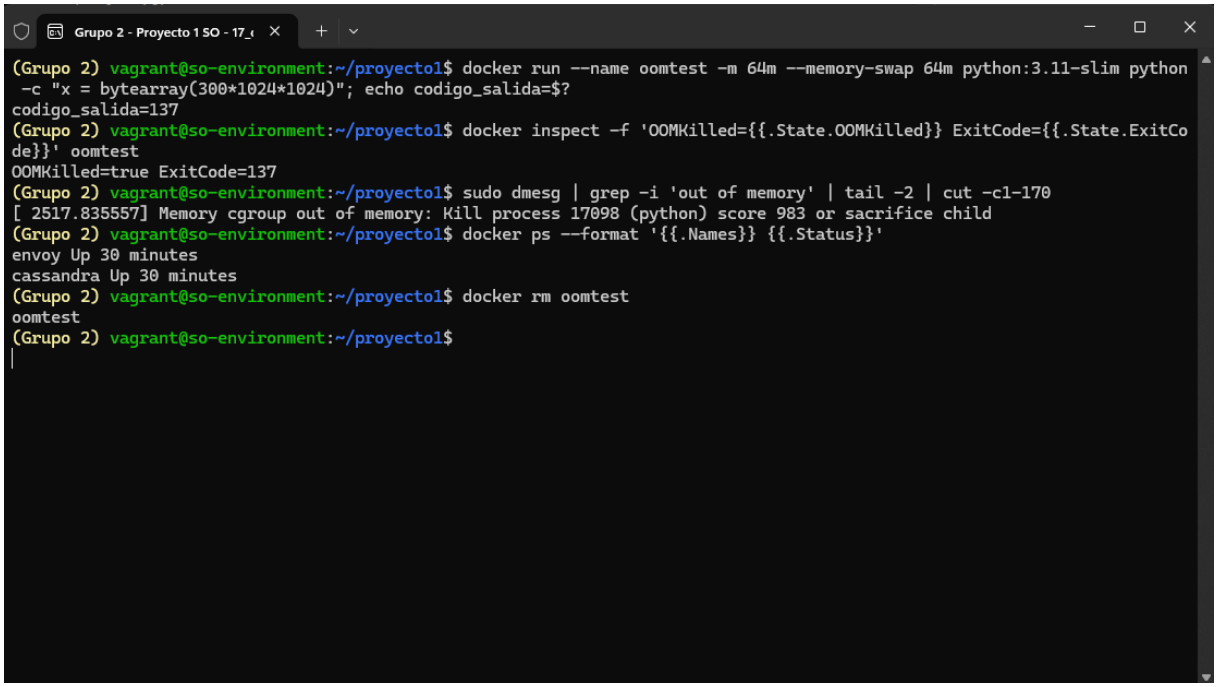



*Figura 21. Contenedor de prueba con 64 MB terminado por el OOM Killer del cgroup.*

## Entregable 2. Justificación técnica

*Pregunta: en un contexto de alta tasa de escrituras entrantes, analice la competencia por CPU y memoria entre el procesamiento de eventos de Envoy y la gestión de heap JVM y memtables en Cassandra dentro de las restricciones del escenario de 6 GB y 3 vCPU.*

### Quién compite por la CPU

Envoy corre un hilo worker por cada unidad de concurrencia, y cada worker atiende todas sus conexiones con un bucle de eventos (epoll) sin bloquearse. Con 0.75 vCPU de cuota y concurrency 1, ese hilo es el único que mueve bytes entre 10 000 clientes y Cassandra. Docker implementa la cuota con el controlador de ancho de banda de CFS: cpu.max vale 75 000 µs cada 100 000 µs, así que si el worker consume 75 ms del periodo, el planificador lo suspende hasta el siguiente periodo y todas las conexiones esperan a la vez.

Cassandra, en cambio, reparte el trabajo en muchos hilos, con 32 de escritura concurrente, hasta 128 de transporte nativo, flushers de memtables, compactación y los hilos del recolector de basura de la JVM. Los 2.25 vCPU de su cuota se los disputan todos. En una carga de solo escrituras, la presión se concentra en el commit log, en el flush de memtables y en las pausas del GC, y cuando el GC consume la cuota los hilos de escritura se retrasan.

En consecuencia, las dos cuotas suman 3.0 vCPU, justo las del host. Por eso el cliente de carga, el daemon de Docker y el kernel compiten con ambos contenedores en cuanto la carga aumenta, y es lo que hace que en la VM del demo el rendimiento varíe tanto entre etapas.

### Quién compite por la memoria

Dentro del contenedor de Cassandra, el heap de 2 GiB es fijo (-Xms = -Xmx), de modo que la JVM no lo agranda ni lo encoge. Las memtables se configuraron como offheap_objects con un tope de 768 MiB; al alcanzarlo, Cassandra vacía a disco la memtable más grande, lo que convierte la presión de memoria en escritura a disco en lugar de en crecimiento sin límite. A eso se suman 384 MiB de caché de archivos, metaspace, pilas de hilos (-Xss256k por hilo) y buffers directos de Netty.

El kernel no lleva la cuenta del heap sino de todas las páginas del cgroup. Cuando el total llega a memory.max, primero recupera páginas de caché de archivos que se puedan descartar (evicción), y si la presión persiste y no hay swap porque memswap_limit es igual a mem_limit, invoca el OOM Killer del cgroup. Este elige un proceso según su oom_score y lo termina con SIGKILL, de ahí el código 137. Como el límite está en el cgroup, el daño queda dentro del contenedor y el host no se ve afectado, como mostró la prueba de 64 MB. Envoy es mucho más liviano (13 MiB en reposo y bajo carga) y su riesgo de memoria es proporcional a las conexiones y a los buffers por conexión. Por eso se fijó per_connection_buffer_limit_bytes en 32 KiB porque, en el peor caso teórico, 10 000 conexiones × 32 KiB × 2 sentidos sumarían unos 625 MiB, algo por encima de los 614 MiB del contenedor. La cifra real debería ser mucho menor porque los buffers se asignan bajo demanda, pero no lo verificamos con 10 000 conexiones y queda como riesgo a vigilar.

### Qué ajustar primero

La respuesta que sostenemos es limitar primero la concurrencia en la entrada. El circuit breaker de Envoy (150 conexiones) es la medida más barata y la única que actúa antes de que el daño ocurra, ya que rechaza el exceso en el borde. Lo comprobamos: con 200 clientes, Envoy dejó pasar 150 y rechazó 50, y Cassandra se mantuvo estable.

El segundo ajuste está dentro de Cassandra y afecta al heap y a memtable_offheap_space, que fijan cuánta memoria se compromete antes de vaciar a disco, y concurrent_writes, que acota los hilos activos. La cuota de memoria del cgroup debe quedar como última red de seguridad, no como mecanismo de control, porque su único efecto es matar el proceso. Se dimensiona con holgura, con 3.90 GB frente a unos 2.5 GiB realmente usados, de modo que solo se active ante un fallo y no en operación normal.

Por último, subir el heap por encima del 50 % de la RAM asignada tampoco ayuda. Cassandra necesita la memoria restante fuera del heap para memtables, caché y sistema de archivos. Con 2 GiB sobre 3.9 GB estamos en el límite razonable.

## Guion de la demo en vivo

Orden de los comandos para la defensa, con lo que se espera ver en cada uno.

**Tabla 4. Secuencia de comandos para la defensa**

| # | Comando | Qué se muestra | Valor esperado |
|---|---|---|---|
| 1 | free -m && nproc | Recursos del host | 5 960 MB, 3 CPU |

| | | | |
|---|---|---|---|
| 2 | docker ps | Servicios activos | envoy y cassandra Up |
| 3 | docker inspect -f '{{.HostConfig.Memory}} {{.HostConfig.NanoCpus}}' envoy cassandra | Límites aplicados | 643825664 / 750000000 y 4186963968 / 2249999872 |
| 4 | cat /sys/fs/cgroup/system.slice/docker-<id>.scope/{memory.max,cpu.max} | Cgroup v2 | 614 MiB y 75000 100000; 3 993 MiB y 224999 100000 |
| 5 | docker stats --no-stream | Uso real frente al límite | Envoy ~13 MiB / 614 MiB; Cassandra ~2.3 GiB / 3.9 GiB |
| 6 | ps aux \| grep -E 'envoy\|java' | Procesos y workers | envoy --concurrency 1; java -Xmx2048M |
| 7 | grep -E 'memtable_offheap\|file_cache\|max_concurrent' cassandra.yaml | Config del motor | 768MiB, 384MiB, 150 |
| 8 | curl -s localhost:9901/server_info \| grep concurrency | Workers de Envoy | 1 |
| 9 | htop | CPU por núcleo y por proceso | 3 núcleos, procesos envoy y java |
| 10 | ./estres.sh | Prueba escalonada | 0 errores, 0 OOM, swap en 0 |
| 11 | python3 conexiones.py | Límite de conexiones | active = 150, overflow = 50 |

## Lista de comprobación

**Tabla 5. Cumplimiento de la rúbrica**

| Criterio de la rúbrica | Cómo se cumplió | Evidencia |
|---|---|---|
| Precisión de cálculos de RAM (20 %) | Overhead 1.50 GB, libre 4.50 GB, Cassandra 3.90 GB, caché 3.19 GB | Tabla 2, figuras 1 y 2 |
| Planificación de CPU y procesos (20 %) | 0.75 y 2.25 vCPU, 1 worker, 10 000 y 150 conexiones | Tabla 2, figuras 10 y 11 |
| Demo en vivo por CLI (30 %) | inspect, cgroup, stats, ps, htop, config y conexiones coinciden con la tabla | Figuras 10 a 14 y 20 |
| Rigor y justificación (20 %) | CFS, cgroups, evicción, OOM Killer y GC con prueba experimental | Sección 7, figuras 20 y 21 |
| Formato y presentación (10 %) | Tabla con vía, unidades y valores calculados y reales | Tablas 2 a 4 |

## Limitaciones

Hay tres limitaciones que conviene declarar. Primera, el rendimiento absoluto de la VM no es representativo: corre bajo Hyper-V con emulación y el cliente de carga ocupa parte de las mismas tres vCPU, por lo que las op/s varían de una etapa a otra. Segunda, no se probaron 10 000 conexiones simultáneas a Envoy; esa cifra sale del cálculo y de la ley de Little, y solo se verificaron 200. Tercera, el cluster es de un solo nodo con factor de replicación 1, así que no se midió ninguna operación de reparación ni de streaming entre nodos.

## Conclusiones

El cálculo se sostiene contra el sistema real. Cada número de la tabla (614 MiB, 3 993 MiB, 0.75 y 2.25 vCPU, 150 conexiones, 1 worker) aparece igual en docker inspect, en el cgroup y en los archivos de configuración, y eso es lo que permite confiar en un dimensionamiento hecho en papel.

Lo que más nos enseñó el ejercicio es que los límites cumplen funciones distintas. La cuota de CPU decide cuánto puede avanzar cada contenedor, el límite de memoria decide cuándo muere, y el circuit breaker de Envoy decide cuánto trabajo entra. El orden en que conviene tocarlos es el inverso al de su costo, de modo que primero va lo que rechaza trabajo, después lo que lo ordena y, al final, lo que mata procesos.

También aprendimos a no atribuir de más. La caída de rendimiento con 64 hilos pudo parecer culpa de Envoy por su único worker, pero la etapa de 128 hilos produjo casi cuatro veces más operaciones por segundo con el mismo worker. Sin esa medición habríamos justificado una causa que los datos no respaldan.

# Anexos

## Anexo A. Comandos ejecutados

**Tabla 6. Comandos principales**

| Paso | Dónde | Comando | Para qué |
|---|---|---|---|
| 1 | Windows | VBoxManage modifyvm vm-sistemas-operativos --memory 6144 --cpus 3 | Ampliar la VM a 6 GB y 3 CPU |
| 2 | Windows | VBoxManage startvm vm-sistemas-operativos --type headless | Encender la VM |
| 3 | Windows | scp docker-compose.yml Dockerfile envoy.yaml monitor.sh estres.sh conexiones.py | Copiar los archivos a ~/proyecto1 |
| 4 | VM | docker pull cassandra:5.0 && docker pull envoyproxy/envoy:v1.31-latest | Descargar imágenes |
| 5 | VM | docker compose build | Construir cassandra5-g2 |
| 6 | VM | docker compose up -d | Levantar Envoy y Cassandra |
| 7 | VM | docker exec cassandra nodetool status | Confirmar nodo UN |
| 8 | VM | docker exec cassandra cqlsh envoy 9042 -e "SELECT cluster_name, release_version FROM system.local" | Consulta a través de Envoy |
| 9 | VM | docker inspect -f ... envoy cassandra | Límites de memoria y CPU |
| 10 | VM | docker stats --no-stream; free -m; ps aux; htop | Estado en reposo |
| 11 | VM | ./estres.sh | Prueba escalonada de 8 a 128 hilos |
| 12 | VM | python3 conexiones.py | Límite de 150 conexiones |
| 13 | VM | docker run --name oomtest -m 64m --memory-swap 64m python:3.11-slim python -c ... | Comprobar el OOM Killer |

## Anexo B. Archivos del proyecto

**docker-compose.yml**

```
services:
  envoy:
    image: envoyproxy/envoy:v1.31-latest
    container_name: envoy
    command: ["-c", "/etc/envoy/envoy.yaml", "--concurrency", "1"]
    volumes:
      - ./envoy.yaml:/etc/envoy/envoy.yaml:ro
    ports:
      - "9042:9042"
      - "9901:9901"
```

```
    mem_limit: 614m
    memswap_limit: 614m
    cpus: 0.75
    ulimits:
      nofile:
        soft: 65536
        hard: 65536
    depends_on:
      - cassandra
  cassandra:
    build: .
    image: cassandra5-g2
    container_name: cassandra
    mem_limit: 3993m
    memswap_limit: 3993m
    cpus: 2.25
    environment:
      MAX_HEAP_SIZE: 2048M
      CASSANDRA_CLUSTER_NAME: grupo2
```

### Dockerfile

```
FROM cassandra:5.0
RUN sed -i -E \
 -e 's/^#? ?memtable_allocation_type:.*/memtable_allocation_type: offheap_objects/' \
 -e 's/^#? ?memtable_offheap_space:.*/memtable_offheap_space: 768MiB/' \
 -e 's/^#? ?file_cache_size:.*/file_cache_size: 384MiB/' \
 -e 's/^#?
?native_transport_max_concurrent_connections:.*/native_transport_max_concurrent_connections: 150/' \
 /etc/cassandra/cassandra.yaml
```

### envoy.yaml

```
admin:
  address:
    socket_address: { address: 0.0.0.0, port_value: 9901 }
static_resources:
  listeners:
    - name: cql
      address:
        socket_address: { address: 0.0.0.0, port_value: 9042 }
      per_connection_buffer_limit_bytes: 32768
      filter_chains:
        - filters:
            - name: envoy.filters.network.tcp_proxy
              typed_config:
                "@type": type.googleapis.com/envoy.extensions.filters.network.tcp_proxy.v3.TcpProxy
                stat_prefix: cassandra_cql
                cluster: cassandra
  clusters:
    - name: cassandra
      type: STRICT_DNS
      connect_timeout: 2s
      circuit_breakers:
        thresholds:
          - max_connections: 150
      load_assignment:
        cluster_name: cassandra
```

```
        endpoints:
          - lb_endpoints:
              - endpoint:
                  address:
                    socket_address: { address: cassandra, port_value: 9042 }
```

### monitor.sh

```
#!/bin/bash
echo "hora  cass_mem cass_cpu% envoy_mem envoy_cpu% conex_envoy_cass libre_MiB swap_MiB"
for i in $(seq 1 ${1:-30}); do
  c=$(docker stats --no-stream --format '{{.MemUsage}} {{.CPUPerc}}' cassandra | awk
'{gsub("MiB","",$1); print $1, $4}')
  e=$(docker stats --no-stream --format '{{.MemUsage}} {{.CPUPerc}}' envoy | awk
'{gsub("MiB","",$1); print $1, $4}')
  n=$(curl -s localhost:9901/stats | awk -F': ' '/^cluster.cassandra.upstream_cx_active:/{print
$2}')
  m=$(free -m | awk '/Mem:/{print $4} /Swap:/{s=$3} END{print s}' | tr '\n' ' ')
  echo "$(date +%H:%M:%S) $c $e $n $m"
done
```

### estres.sh

```
#!/bin/bash
mkdir -p logs
for t in 8 32 64 128; do
  echo "== etapa: $t hilos de escritura =="
  docker run --rm --network proyecto1_default -m 400m --cpus 0.5 -e JVM_OPTS=-Xmx256m --entrypoint
/opt/cassandra/tools/bin/cassandra-stress cassandra:5.0 write duration=40s -rate threads=$t -node
envoy -mode native cql3 > logs/stress_$t.log 2>&1 &
  sleep 15
  ./monitor.sh 4 | tee -a logs/monitor.log
  wait
  grep -E "^(Op rate|Latency mean|Latency 99th|Total errors)" logs/stress_$t.log | tail -4
  oom=$(docker inspect -f '{{.State.OOMKilled}}' cassandra envoy | tr '\n' ' ')
  echo "OOMKilled cassandra/envoy: $oom"
  pct=$(docker stats --no-stream --format '{{.MemPerc}}' cassandra | tr -d '%' | cut -d. -f1)
  if [ "$pct" -ge 95 ]; then echo "memoria al ${pct}%: se detiene la prueba"; break; fi
  sleep 10
done
```

### conexiones.py

```
import os, socket, time
socks = []
for objetivo in (50, 100, 150, 200):
    while len(socks) < objetivo:
        try:
            socks.append(socket.create_connection(("127.0.0.1", 9042), timeout=3))
        except OSError as e:
            print("error:", e)
            break
    time.sleep(6)
    print("== conexiones abiertas por el cliente:", len(socks))
    os.system("curl -s localhost:9901/stats | grep -E
'^cluster.cassandra.upstream_cx_(active|overflow):'")
time.sleep(10)
print("== 10 s despues, con 200 conexiones del cliente aun abiertas")
```

```
os.system("curl -s localhost:9901/stats | grep -E
'^cluster.cassandra.upstream_cx_(active|overflow):'")
```